PokeChess Notation Design: FEN & PGN Equivalents

Status: Proposal — pre-implementation
Authors: nherringer
Audience: ML bot team + frontend/async team

Background

This document answers the question: *does PokeChess need a FEN or PGN equivalent, and if so, what does that look like?*

The short answer is **yes to both**, for three confirmed use cases:

1. **Frontend/async protocol** — passing game state between the ML bot and the frontend layer
2. **Save/resume** — persisting a game mid-session and reloading it
3. **Game replay and analysis** — logging moves so a game can be replayed or analyzed after the fact

---

Why the Zobrist Hash Is Not Enough

The Zobrist hash is a 64-bit integer used as a lookup key in the MCTS transposition table. It is **intentionally lossy** — designed for fast approximate lookups, not for unique state identification.

What the Zobrist hash bins or omits

State field	What Zobrist does
HP per piece	Bins at `hp // 50` — 6 buckets cover 0–250+ HP, so e.g. 110 HP and 140 HP hash the same
Piece inside a Safetyball	**Not hashed at all**
`foresight_used_last_turn` flag	**Not hashed** — prevents consecutive Foresight but is invisible to the TT
`has_traded` flag	**Not hashed** — free-action gate per turn

This means two distinct positions can share a Zobrist hash, and a hash cannot be inverted to reconstruct state. It is the wrong tool for serialization.

What Full State Consists Of

For reference, here is every field that a complete position snapshot must capture:

Per Piece

Field	Type	Notes
`piece_type`	enum (16 values)	Includes evolved forms (Raichu, Vaporeon, etc.)
`team`	RED / BLUE	
`row`, `col`	0–7 each	Board position
`current_hp`	int, multiples of 10	Up to 250+ depending on piece
`held_item`	enum (6 values)	NONE, WATERSTONE, FIRESTONE, LEAFSTONE, THUNDERSTONE, BENTSPOON
`stored_piece`	Optional[Piece]	Nested piece inside a Safetyball; `None` if empty

Per GameState

Field	Type	Notes
`active_player`	Team	Whose turn
`turn_number`	int	Increments each half-move
`pending_foresight`	dict[Team → ForesightEffect?]	Delayed attack per team

`foresight_used_last_turn`	dict[Team → bool]	Prevents consecutive Foresight
`has_traded`	dict[Team → bool]	Free-action gate for item trades

ForesightEffect

Field	Type
`target_row`, `target_col`	0–7
`damage`	int (always 120 for Mew/Espeon)
`resolves_on_turn`	absolute turn number

Existing Serialization to Build On

The C++ bridge codec (`cpp/state_codec.py`) already encodes **every field listed above** into a binary wire format. It handles:

- All piece fields including stored pieces inside Safetyballs
- Both foresight effects
- `has_traded` and `foresight_used_last_turn` flags

This is a reference implementation. The gap is that it is binary/process-internal — not a portable, human-readable format. The proposal below is to surface it as structured text.

Proposal: Two Separate Primitives

Primitive 1 — PokeChess-FEN (position snapshot)

Purpose: complete, lossless encoding of a game state at a point in time. Used for save/resume and as the state handoff format in the frontend protocol.

Approach: JSON serialization via `to_dict()` / `from_dict()` methods on `Piece`, `ForesightEffect`, and `GameState`.

Unlike chess FEN (which uses a compact hand-crafted text syntax optimized for a narrow domain), PokeChess state has too many numeric fields — HP values, turn numbers, damage integers — to benefit from custom encoding. A structured JSON object is:

- Easier to read in logs and debuggers
- Straightforward to extend as rules evolve
- Trivially consumed by a frontend (JSON is the native async protocol format anyway)

If a compact opaque token is ever needed (e.g. as a URL parameter), the binary codec output can be base64-encoded — it already round-trips correctly.

Files to modify:

- `engine/state.py` — add `to_dict()` / `from_dict()` to `Piece`, `ForesightEffect`, `GameState`

Estimated work: 2–3 hours. Pure addition, no existing logic changes.

Example output (abbreviated):

```
{
  "active_player": "RED",
  "turn_number": 14,
  "has_traded": {"RED": false, "BLUE": false},
  "foresight_used_last_turn": {"RED": false, "BLUE": true},
  "pending_foresight": {
    "BLUE": {"target_row": 4, "target_col": 3, "damage": 120, "resolves_on_turn": 15}
  },
  "board": [
    {"piece_type": "SQUIRTLE", "team": "RED", "row": 3, "col": 2,
     "current_hp": 160, "held_item": "WATERSTONE", "stored_piece": null},
    {"piece_type": "SAFETYBALL", "team": "BLUE", "row": 5, "col": 4,
     "current_hp": 0, "held_item": "NONE",
     "stored_piece": {"piece_type": "EEVEE", "team": "BLUE",
                      "row": 5, "col": 4, "current_hp": 80,
                      "held_item": "THUNDERSTONE", "stored_piece": null}},
    "..."
  ]
}
```

Primitive 2 — PokeChess-PGN (move log)

Purpose: a sequence of moves that can reproduce a game from the standard starting position. Used for replay, analysis, and game sharing.

The key difference from chess PGN: pokeball ATTACK is stochastic (50% capture / 50% fail). The outcome must be recorded in the log, or replay is non-deterministic.

Proposed notation: `<from> <action> <to> [/<secondary>] [*<slot>] [!<outcome>]`

Component	Meaning

`<from>`	`<col><row>` in algebraic style, e.g. `e2`
`<action>`	`MV` (move), `AT` (attack), `PB` (pokeball), `FS` (foresight), `TR` (trade), `EV` (evolve), `QA` (quick attack), `RL` (release)
`<to>`	destination or target square
`/<secondary>`	for QUICK_ATTACK: the post-attack move square
`*<slot>`	for Mew attack type (0–2) or Eevee evolution choice (0–4)
`!<outcome>`	for pokeball only: `C` (captured) or `F` (failed)

Examples:

```
e2 MV e4              -- Squirtle slides from e2 to e4
d4 AT c5              -- deterministic attack
b3 PB d5!C            -- Stealball thrown at d5, capture succeeded
b3 PB d5!F            -- Stealball thrown at d5, failed
e1 QA e3/e5           -- Eevee: Quick Attack e3, then move to e5
d5 FS e7              -- Foresight targeting e7
g1 EV*2               -- Eevee evolves to Leafeon (slot 2)
h2 TR g2              -- Item trade with piece at g2
c3 RL                 -- Release stored Pokémon from Safetyball

```

`GameRecord` **class** to add to `engine/`:

- Wraps a `GameState`, records each `(move, outcome_index)` pair as the game is played
- `to_pgn_str()` → text header + move list
- `from_pgn_str()` → replays from `GameState.new_game()`, yielding the final state and full history

Files to add/modify:

- `engine/notation.py` (new file) — `move_to_str()`, `str_to_move()`, `GameRecord` class
- `tests/test_notation.py` (new file) — round-trip and replay tests

Estimated work: 4–6 hours. Medium complexity, primarily due to stochastic outcome recording and the QUICK_ATTACK secondary-move encoding.

What to Skip

- **Adapting chess FEN text syntax** — the field set is too different; a hand-crafted encoding adds maintenance burden with no benefit over JSON
- **Go SGF** — irrelevant domain
- **Upgrading Zobrist to cover missing fields** — orthogonal concern; the hash is an optimization for the transposition table, not a state representation. Changing its resolution would shift MCTS statistics, not fix serializability.

Implementation Order

1. **PokeChess-FEN first** (`engine/state.py` changes only). This unblocks the frontend protocol and save/resume immediately, and is a prerequisite for PGN replay.
2. **PokeChess-PGN second** (`engine/notation.py`) once FEN is stable.

Verification

Test	Method
FEN round-trip	`GameState.from_dict(state.to_dict()) == state` for mid-game states with stored pieces, pending Foresight, held items
PGN replay	Record a fixed game → serialize → replay → final state matches original
Stochastic replay	Record a game with both a pokeball capture and a pokeball fail → both replay correctly
Frontend integration	ML bot serializes state via `to_dict()`; frontend deserializes and renders